MNHEME — Memory Generator

Genera ricordi sintetici realistici tramite un LLM locale e li posta direttamente sulla REST API di MNHEME.

Requisiti

- Python 3.10+
- Un server LLM locale attivo con endpoint OpenAI-compatibile (LM Studio, Ollama, llama.cpp, vLLM, ecc.)
- API MNHEME in esecuzione (`uvicorn mnheme_api:app --port 8000`)
- **Nessuna dipendenza esterna** — usa solo la stdlib Python (`urllib`, `json`, `concurrent.futures`)

Installazione

```bash
# Nessun pip install necessario — solo stdlib Python
# Il file deve stare nella stessa directory di mnheme.py
# (per importare automaticamente i Feeling validi)

python generate_memories.py --help
```

Utilizzo rapido

```bash
```

```
# Genera 20 ricordi con modello auto-rilevato
python generate_memories.py --n 20

# Genera 50 ricordi su server non-default
python generate_memories.py --n 50 \
  --llm-url http://localhost:1234 \
  --api-url http://localhost:8000

# Dry-run: genera e valida senza postare nulla
python generate_memories.py --n 10 --dry-run --verbose

# Generazione massiccia con più parallelismo
python generate_memories.py --n 200 --batch-size 8 --workers 5
```

Opzioni CLI

Flag	Default	Descrizione
`--n`	`10`	Numero totale di ricordi da generare
`--llm-url`	`http://localhost:1234`	Base URL del server LLM
`--model`	*(auto)*	Nome del modello — se omesso viene rilevato via `/v1/models`
`--api-key`	*(vuoto)*	API key per il server LLM (opzionale per istanze locali)
`--api-url`	`http://localhost:8000`	URL base dell'API MNHEME
`--batch-size`	`5`	Ricordi richiesti per chiamata LLM (max 10)
`--workers`	`3`	Thread paralleli per i batch
`--dry-run`	off	Genera e valida senza postare sull'API
`--verbose`	off	Stampa il campo `content` completo di ogni ricordo
`--sequential`	off	Disabilita la generazione parallela (utile per debug)

Come funziona

Pipeline per ogni batch

```
1. _build_prompt()     → costruisce prompt con seed random,
                         domini tematici, concetti/feeling da evitare

2. LLMClient.complete() → chiamata HTTP a /v1/chat/completions
                          temperature: ~0.95–1.10 (variabile per diversità)
                          max_tokens: 2048

3. _parse_memories()   → 3 tentativi di parsing JSON:
                          - parse diretto
                          - strip code fences (```)
                          - regex per estrarre il primo array valido

4. _validate_record()  → normalizza ogni campo:
                          concept  → prima parola, minuscolo, solo alfanumerici
                          feeling  → fuzzy fix + fallback random se invalido
                          content  → troncato a 400 caratteri
                          tags     → lista di stringhe, cap a 6

5. MnhemeClient.post() → POST /memories con retry backoff esponenziale
```

Parallelismo

I batch vengono eseguiti in parallelo con `ThreadPoolExecutor`. Ogni batch riceve uno snapshot dei concetti/feeling già usati per minimizzare i duplicati. La dedup locale all'interno di ogni batch è garantita; tra batch paralleli diversi può esserci qualche sovrapposizione (accettabile a fronte delle performance).

Per generazione sequenziale con dedup perfetta usare `--sequential`.

Retry e fault tolerance

- **LLM**: 3 tentativi con backoff `2^n` secondi su errori di rete o JSON malformato
- **API POST**: 3 tentativi con backoff `1.5^n` secondi su errori HTTP
- Un batch che fallisce completamente viene segnalato ma non interrompe gli altri

---

Output terminale

```
MNHEME — Memory Generator  v2.0
──────────────────────────────────────────────────────────

  → Auto-discovery modello da http://localhost:1234… nvidia/nemotron-mini
```

```
  → Ping MNHEME API http://localhost:8000… ok

  Generando 20 ricordi in 4 batch da 5 · 3 worker paralleli
──────────────────────────────────────────────────────────
  ✓ mutuo                  ansia       id: a3f9bc12-…  [casa, 2019, banca]
  ✓ licenziamento          paura       id: b7e2d401-…  [lavoro, 2022]
  ✓ tradimento             rabbia      id: c1a8f033-…  [amicizia]
  …

──────────────────────────────────────────────────────────
  Completato in 18.4s
  20 postati  ·  0 falliti  ·  0 saltati

  Concetti generati: dentista, licenziamento, mutuo, pensione, …

  Distribuzione emotiva:
    nostalgia            ▬▬▬ 4
    ansia                ▬▬ 3
    gioia                ▬▬ 3
    …
```

Qualità della generazione

Il prompt inviato al modello include:

- **Seed casuale** per ogni batch — evita risposte cached/identiche
- **22 domini tematici** (lavoro, famiglia, denaro, corpo, viaggi, perdita…) campionati a rotazione
- **Lista nera di concetti** già usati — il modello non li ripete
- **Dedup feeling** — il modello evita di saturare un singolo stato emotivo
- **Regole esplicite anti-cliché** per il campo `content`
- **Formato note** con contesto concreto (luogo, data, persone)

Feeling disponibili

I 26 valori dell'enum `Feeling` di MNHEME:

```
ansia · paura · sollievo · tristezza · gioia · rabbia · vergogna
senso_di_colpa · nostalgia · speranza · orgoglio · delusione
solitudine · confusione · gratitudine · invidia · imbarazzo
eccitazione · rassegnazione · stupore · amore · malinconia
serenità · sorpresa · noia · curiosità
```

Se `mnheme.py` non è nel path, il generatore usa questo elenco come fallback hardcoded.

Compatibilità LLM

Testato con:

Server	Note
LM Studio	Auto-discovery via `/v1/models`, nessuna API key necessaria
Ollama	`--llm-url http://localhost:11434`
llama.cpp server	`--llm-url http://localhost:8080`
vLLM	`--llm-url http://localhost:8000 --api-url http://localhost:8001`
Groq / Mistral / OpenRouter	`--llm-url <endpoint> --api-key <key>`

Modelli consigliati

Per la qualità del JSON generato, modelli istruzione con almeno 7B parametri danno risultati migliori. Con modelli più piccoli (3B) usare `--batch-size 3` per ridurre la probabilità di JSON troncato.

Esempi avanzati

```bash
```

```python
# 100 ricordi su Ollama con llama3
python generate_memories.py \
  --n 100 \
  --llm-url http://localhost:11434 \
  --model llama3.1:8b \
  --batch-size 6 \
  --workers 4

# Dry-run verbose per testare la qualità del modello
python generate_memories.py --n 5 --dry-run --verbose --sequential

# Popola un database remoto MNHEME
python generate_memories.py \
  --n 50 \
  --api-url https://my-mnheme-instance.example.com \
  --llm-url http://localhost:1234

# Debug di un singolo batch
python generate_memories.py --n 3 --sequential --verbose
```

Struttura del codice

```
generate_memories.py
│
├── MemoryRecord          dataclass con validazione lazy
├── LLMClient             client HTTP stdlib (no openai SDK)
│   ├── discover_model()  GET /v1/models → primo modello disponibile
│   └── complete()        POST /v1/chat/completions
├── MnhemeClient          client REST per l'API MNHEME
│   ├── ping()            GET /stats
│   └── post()            POST /memories con retry
│
├── _build_prompt()       costruisce il prompt con seed, domini, blacklist
├── _parse_memories()     parsing JSON robusto a 3 tentativi
├── _validate_record()    normalizzazione e sanitizzazione dei campi
└── _generate_batch()     orchestrazione genera→valida→posta con retry
```